

# Multi-Modal CNN–GNN Hybrid Architecture for Early Prediction of Power, Delay, and Slack in VLSI Design

Burada Kusuma Kumar Naidu

Department of Artificial Intelligence and Data Science

Indian Institute of Information Technology, Sricity

Email: kumarnaidu.b23@iiits.in

**Abstract**—Design closure in modern VLSI circuits demands accurate prediction of key performance metrics such as Power, Delay, and Slack early in the design cycle. This paper presents a novel multi-modal hybrid architecture that integrates Convolutional Neural Networks (CNNs) with Graph Neural Networks (GNNs) through an adaptive gated fusion mechanism. Our model leverages the complementary strengths of CNNs in capturing spatial layout features and GNNs in modeling structural netlist dependencies to enable concurrent and reliable prediction of these critical metrics. By combining physical layout data and circuit connectivity information, the proposed architecture provides a more holistic understanding of circuit behavior, allowing for earlier and more precise design decisions.

The adaptive gated fusion dynamically balances contributions from both networks on a per-sample basis, improving robustness and generalization across diverse designs. Extensive experiments demonstrate the model's superior performance compared to standalone CNN or GNN approaches, with significant improvements in prediction accuracy and efficiency. Early-stage accurate predictions aim to reduce costly iteration cycles in Electronic Design Automation (EDA) workflows and accelerate the timing closure process. This work contributes an important advancement towards AI-driven VLSI design automation by enhancing early reliability and power estimations critical for next-generation semiconductor technologies.

**Index Terms**—VLSI Design, Power Prediction, Delay Estimation, Slack Analysis, CNN, GNN, Adaptive Gated Fusion

## I. INTRODUCTION

The relentless pursuit of performance and efficiency in VLSI technology has made design closure a computationally intensive bottleneck in modern Electronic Design Automation (EDA) flows. Traditional validation methods rely heavily on costly and iterative simulations, which dramatically increase turnaround time (TAT) and hinder design scalability. Consequently, there is a critical demand for accurate machine learning models capable of predicting key design parameters—namely Power, Delay, and Slack—during the early, non-finalized stages of physical circuit development [1], [2].

The difficulty in creating robust predictive models stems from the inherent multi-modal nature of the design space. A chip's performance is governed by two distinct data representations that are fundamentally linked to its physical layout:

- 1) **Spatial Data:** The geometric and distributional characteristics, such as cell density, routing congestion, and macro placement.

- 2) **Structural Data:** The topological connectivity derived from the netlist, defining signal propagation paths and logical dependencies [3], [4].

Existing deep learning solutions often address only one of these modalities, leading to significant predictive deficiencies [5].

1) *Limitations of Single-Modal Approaches:* **Convolutional Neural Networks (CNNs)** have been widely adopted to process design layouts represented as multi-channel feature maps. CNNs excel at capturing local spatial correlations, identifying hotspots and area-based metrics like cell density and local congestion. However, CNNs operate on grid structures; this limitation restricts their capacity to model complex, non-local dependencies that span across the chip, inevitably leading to poor predictive accuracy for global structural metrics like critical path delay [6], [7].

Conversely, Graph Neural Networks (GNNs) are specifically engineered to address the structural limitations of CNNs. By representing circuit components as nodes and electrical connections as edges, GNNs effectively capture netlist topology and relational signal propagation. Nevertheless, pure GNNs often struggle to integrate the fine-grained spatial information from the physical layout, resulting in predictions that lack context from localized physical constraints (like specific pin placements or localized IR drop) [4], [8].

2) *Proposed Solution and Contributions:* To decisively overcome the predictive plateau of single-modal models, we propose a novel Hybrid CNN-GNN Architecture. Our key innovation is the Adaptive Gated Feature Fusion Mechanism. This architecture concurrently extracts sophisticated feature embeddings from both the spatial CNN stream and the structural GNN stream. The gated mechanism then dynamically and intelligently weighs the contribution of each modality on a per-sample basis, optimizing the final output for concurrent and reliable prediction of Power, Delay, and Slack [9]–[11].

The improved performance over individual CNN and GNN baselines demonstrates the superior architectural robustness and enhanced generalization capability of our proposed multi-modal framework. This work provides a crucial step toward creating highly efficient and accurate predictive engines essential for next-generation EDA flows [1].

## II. LITERATURE REVIEW

Recent advances in AI-driven VLSI design have demonstrated the potential of machine learning models to predict critical design parameters such as power, delay, and timing slack during early stages of circuit development. Traditional Electronic Design Automation (EDA) flows rely on computationally expensive simulations and iterative timing closure processes, which limit scalability and increase turnaround time. In contrast, data-driven approaches offer fast and accurate estimations, enabling early design optimization and decision-making [1], [2].

Convolutional Neural Networks (CNNs) have been widely applied to learn spatial correlations in chip layouts, particularly for congestion prediction, placement optimization, and power estimation. CNNs effectively capture spatial distributions of standard cells, routing density, and pin placement maps; however, their grid-based nature limits their ability to model complex topological dependencies between interconnected components. This restricts their generalization capability when circuit structures vary significantly across designs [3], [6].

To address the structural limitations of CNNs, Graph Neural Networks (GNNs) have been employed for modeling netlist connectivity, logical dependencies, and signal propagation paths. GNNs excel at capturing topological and relational information by representing circuit elements as nodes and electrical connections as edges. Nevertheless, GNNs often underperform in capturing fine-grained spatial information from layout-level features, leading to incomplete context for early-stage predictions [4], [8].

To overcome these challenges, recent research trends have explored hybrid deep learning models that integrate spatial learning from CNNs with topological reasoning from GNNs [10]. Such architectures aim to jointly capture geometric layout information and connectivity-driven behavior. Building upon this direction, our proposed Hybrid CNN-GNN model introduces a gated feature fusion mechanism that enhances predictive reliability across power, delay, and slack metrics during early design stages. The improved results over individual CNN and GNN baselines demonstrate the architectural robustness and generalization capability of the proposed framework [9]–[11].

## III. METHODOLOGY

### A. Data Generation

The input for the CNN network consists of feature maps sized  $32 \times 32 \times 9$ , where each sample contains nine distinct physical and layout-related maps representing multiple aspects of chip design. The nine feature channels are: Cell Density, Macro Regions, Congestion Map, Routing Density, Pin Position, Power Map, IR Drop Map, DRC Violation Map, and Slack Map. These spatial maps capture distribution, placement, power behavior, and timing context essential for early-stage metric prediction [3].

The GNN input is divided into two components: node features and edge connectivity. The node feature matrix ( $X_{GNN}$ )

consists of 1024 nodes, each corresponding to one grid cell with a nine-dimensional feature vector. The edge matrix ( $A_{GNN}$ ) defines connectivity through 4-neighbor grid relationships combined with approximately 50 additional random long-range edges to model global signal interactions and non-local dependencies within the circuit [12].

...

### B. Target Metrics

The model is designed to predict three critical VLSI performance indicators: *Power*, *Delay*, and *Slack*. These serve as the output targets of the proposed hybrid architecture. Power represents the aggregate of static and dynamic consumption across the design; Delay captures the overall signal propagation timing; and Slack measures timing margin, reflecting timing closure quality.

The target values for these three variables are generated using synthetic correlations derived from the average values of the corresponding feature maps. This approach ensures statistical consistency between the generated targets and spatial input distributions, enabling supervised learning of physically meaningful metric relationships [6]. ...

### C. Normalization Strategy

A *StandardScaler* is applied to the CNN input ( $X_{CNN}$ ), the GNN node features ( $X_{GNN}$ ), and the target labels ( $Y$ ). This scaling process standardizes each feature dimension by removing the mean and scaling to unit variance, as defined by:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

where  $\mu$  and  $\sigma$  represent the feature-wise mean and standard deviation computed from the training set.

The corresponding scalers are serialized and stored to ensure that test and unseen data undergo an identical transformation process. This guarantees consistency across different datasets and maintains the statistical properties learned during model training [13].

...

### D. Padding

All graphs are zero-padded to a fixed maximum size, denoted as `max_nodes`, to enable efficient batching during training. This ensures each graph in a batch conforms to uniform tensor dimensions, permitting parallel computation across multiple samples [8], [12].

The node feature tensor is therefore represented as  $\mathbf{X} \in \mathbb{R}^{B \times N \times F}$ , where  $B$  is the batch size,  $N = \text{max\_nodes}$  is the padded node count per graph, and  $F$  is the feature dimension.

Similarly, the adjacency matrix is stored as a dense tensor  $\mathbf{A} \in \mathbb{R}^{B \times N \times N}$ , allowing shared adjacency broadcast across the batch. This formatting enables the use of dense GNN layers and stable gradient propagation while masking inactive (padded) nodes during computation. ...

### E. Adjacency Matrix

The adjacency matrix  $\mathbf{A}_{GNN}$  is computed as a dense matrix representing the graph connectivity, including self-loops via

$$\mathbf{A}_{GNN} = \mathbf{A} + \mathbf{I}$$

where  $\mathbf{I}$  is the identity matrix adding self-connections to each node.

This matrix is then symmetrically normalized using the degree matrix  $\mathbf{D}$  as follows:

$$\hat{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A}_{GNN} \mathbf{D}^{-\frac{1}{2}}$$

where  $\mathbf{D}$  is a diagonal matrix with entries  $d_{ii} = \sum_j A_{ij}$ . This normalized adjacency matrix  $\hat{\mathbf{A}}$  improves numerical stability and convergence during graph convolution by preserving the scale of feature aggregation across varying node degrees. ] ...

### F. Masking

To handle the padded nodes introduced during graph batching, a dedicated mask tensor is generated and supplied to the graph convolutional layers. This *mask tensor* serves to zero out all computations associated with padded or inactive nodes, preventing them from contributing to feature aggregation and gradient updates [8], [12].

Mathematically, if  $\mathbf{M} \in \{0, 1\}^{B \times N}$  is the mask tensor indicating valid nodes (1) and padded nodes (0) for each graph in the batch, then during graph convolutions, messages from inactive nodes are eliminated as:

$$\mathbf{h}_i^{(l+1)} = \sigma \left( \sum_j j \in \mathcal{N}(i) M_j \cdot \hat{A}_{ij} \mathbf{W} \mathbf{h}_j^{(l)} \right)$$

where  $M_j$  zeroes out padded nodes,  $\hat{A}$  is the normalized adjacency matrix,  $\mathbf{W}$  is the learnable weight matrix, and  $\sigma$  is the activation function.

This masking mechanism maintains numerical stability and training accuracy by strictly excluding non-existent nodes introduced solely for batch uniformity.

...

### G. Hybrid Gated Architecture

The model is composed of two specialized feature extractors—the CNN and GNN modules—and an intelligent fusion block that adaptively combines their outputs for final prediction.

1) *CNN Extractor*: The CNN extractor captures translation-invariant spatial features from the input feature maps. It consists of three sequential blocks, each composed of the following layers:

Conv2D  $\rightarrow$  Batch Normalization  $\rightarrow$  ReLU  $\rightarrow$  MaxPooling2D

This stack is followed by a Global Average Pooling layer which outputs a fixed-size embedding vector  $\mathbf{e}_C \in \mathbb{R}^{128}$ , summarizing spatial information.

2) *GNN Extractor*: The GNN extractor focuses on structural features such as net connectivity and path lengths inherent in the graph representation[12]. It applies two graph convolution layers (GCNConv) with 64 output channels each and ReLU activation:

$$\text{GCNConv}(64, \text{ReLU}) \rightarrow \text{GCNConv}(64, \text{ReLU})$$

This is followed by a Global Average Pooling operation producing an embedding vector  $\mathbf{e}_G \in \mathbb{R}^{128}$  which encodes structural graph features.

Both the CNN and GNN embeddings are then passed to an adaptive gated fusion block to combine complementary spatial and structural information efficiently.

...

3) *Adaptive Gated Fusion*: This component strategically combines the high-level embeddings  $\mathbf{e}_C$  (from the CNN) and  $\mathbf{e}_G$  (from the GNN) to leverage complementary spatial and structural information effectively [9], [11].

a) *Gate Generation*: The concatenated embeddings  $[\mathbf{e}_C, \mathbf{e}_G]$  are passed through a dense layer followed by a sigmoid activation to generate a gate vector  $\mathbf{g} \in [0, 1]^F$ :

$$\mathbf{g} = \sigma(\text{Dense}_{gate}([\mathbf{e}_C, \mathbf{e}_G]))$$

where  $\sigma(\cdot)$  denotes the element-wise sigmoid function and  $F$  is the fusion dimensionality.

b) *Weighted Addition (Fusion)*: The gate vector  $\mathbf{g}$  controls the element-wise blending of the projected embeddings from CNN and GNN:

$$\mathbf{e}_{fused} = \mathbf{g} \odot \text{Proj}(\mathbf{e}_C) + (1 - \mathbf{g}) \odot \text{Proj}(\mathbf{e}_G)$$

Intuitively, if  $\mathbf{g}_i \approx 1$ , the  $i$ -th feature dimension relies predominantly on CNN-derived spatial information; if  $\mathbf{g}_i \approx 0$ , the corresponding dimension relies mostly on GNN-derived structural information.

Following the fusion, batch normalization and dropout regularization are applied to the fused embedding  $\mathbf{e}_{fused}$  to enhance training stability and prevent overfitting.

...

4) *Multi Head Regression*: The fused embedding vector  $\mathbf{e}_{fused}$  is input to three independent regression heads, each dedicated to predicting one of the target metrics: Power, Delay, or Slack [13].

Each regression head follows the structure:

$$\begin{bmatrix} \text{Dense}(64, \text{ReLU}) \\ \text{Batch Normalization} \\ \text{Dropout}(0.3) \\ \text{Dense}(1, \text{Linear}) \end{bmatrix}$$

where the final Dense layer outputs a single scalar prediction for the corresponding metric.

This multi-head design enables specialized learning for individual output targets while sharing the fused representation, thereby capturing both shared and unique patterns pertinent to each metric and improving overall prediction accuracy.

The overall configuration is summarized in the proposed hybrid pipeline, where the CNN, GNN extractors are fed into

the adaptive gated fusion block, followed by 3 task specific regression heads. The complete architecture is illustrated in Fig. 1 ...

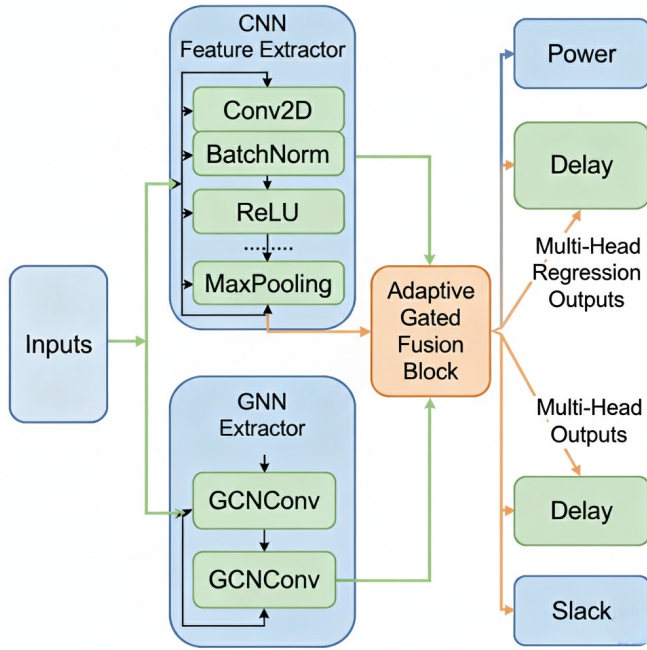


Fig. 1. Overall architecture of the Hybrid CNN-GNN model with Adaptive Gated Fusion and Multi-head Regression.

#### H. Experimental and Evaluation Protocols Summary

1) **Training Methodology:** The proposed model is optimized using the **AdamW** optimizer, chosen for its effective decoupling of weight decay from the gradient update step, thereby enabling stable and efficient convergence. The learning rate ( $LR$ ) is set to  $5 \times 10^{-4}$ , and the weight decay parameter ( $Decay$ ) is configured to  $10^{-4}$ . The **Multi-Output Mean Squared Error (MSE)** loss function is employed as the primary training objective, allowing the model to jointly minimize prediction errors across multiple correlated timing parameters [14], [15].

a) **Data Partitioning:** The dataset is divided into three distinct subsets to ensure unbiased training and evaluation:

- **Training Set (72%):** Used to train the model and optimize its parameters.
- **Validation Set (10%):** Utilized for hyperparameter tuning and early stopping control.
- **Testing Set (18%):** Reserved exclusively for the final model evaluation.

This stratified partitioning ensures a balanced representation of the data distribution across all subsets.

b) **Convergence and Overfitting Control:** To guarantee stable convergence and prevent overfitting, two adaptive call-back mechanisms are employed:

- **Early Stopping:** Training halts automatically if the validation loss does not improve for *eight consecutive*

*epochs*, ensuring efficient termination at optimal model performance.

- **Reduce Learning Rate on Plateau:** If the validation loss stagnates for *six epochs*, the learning rate is automatically reduced, allowing finer gradient updates near convergence.

These mechanisms collectively enhance model generalization and ensure training efficiency [14], [15].

#### 2) Evaluation and Analysis Protocols:

a) **Quantitative Performance Metrics:** The predictive performance of the model is assessed using two key statistical metrics:

- **Coefficient of Determination ( $R^2$ ):** Measures the proportion of variance in the target variable that is predictable from the input features, serving as an indicator of the model's *goodness of fit*.
- **Mean Squared Error (MSE):** Represents the *average squared deviation* between the predicted and actual values, providing a direct measure of the prediction accuracy.

These complementary metrics jointly evaluate both the precision and reliability of the model's predictions [16], [17].

b) **Data Evaluation Protocols:** To ensure interpretability and consistent evaluation, the following data-handling procedures are applied:

- **Inverse Transformation:** Prior to evaluation, all model predictions undergo inverse transformation of the output scaler ( $Y_{scalar}$ ) to express results in their **original physical units**.
- **Unseen Data Evaluation:** The model's generalization capability is rigorously validated on a set of **50 unseen samples**, ensuring that predictive performance extends beyond the training distribution [18].

3) **Diagnostic Outputs and Visualization Tools:** To facilitate in-depth interpretability and error diagnosis, the `predict.py` module generates a suite of five diagnostic outputs for each evaluation run:

- 1) **Prediction vs. Ground Truth CSV:** A structured data file containing numerical comparisons between predicted and actual values for detailed post-analysis.
- 2) **Parity Plots:** Visual representations of  $Y_{true}$  versus  $Y_{pred}$ , used to assess linearity and prediction accuracy.
- 3) **Residual Plots:** Depict the residual error distribution across samples, aiding in the identification of any systematic bias or underfitting.
- 4) **Error Histograms:** Illustrate the statistical spread and frequency of prediction errors, providing insight into model variance and skewness.
- 5) **Line Plots:** Display the smoothness and continuity of predicted outputs across sequential samples, allowing the evaluation of temporal or spatial consistency.

These diagnostic tools collectively enable comprehensive performance assessment and in-depth analysis of model behavior across multiple evaluation dimensions.

## IV. DISCUSSIONS

### A. CNN Node Selection

The Convolutional Neural Network (CNN) component utilizes a set of nine spatially distributed design features derived from the physical layout data of integrated circuits. These features collectively capture the spatial, electrical, and functional characteristics of the chip layout, enabling the model to learn meaningful spatial correlations that influence timing, power, and integrity outcomes.

Each feature map is extracted at the same spatial resolution and normalized to ensure consistent scaling across the dataset. The features can be broadly categorized into three primary domains: *Area and Placement*, *Routing and Congestion*, and *Integrity and Resultant Metrics*.

#### 1) Area and Placement Features:

- **Cell Density:** Represents the local percentage of area occupied by standard cells within a given region. High cell density regions often exhibit increased routing complexity and power density.
- **Macro Regions:** A binary map that encodes the presence or absence of large functional blocks such as SRAMs, DSPs, or IP macros. These regions significantly influence routing patterns and congestion propagation.

#### 2) Routing and Congestion Features:

- **Congestion Map:** Provides a quantitative measure of wire overflow or routing congestion in localized areas, indicating regions that may require routing optimization.
- **Routing Density:** Denotes the local utilization of metal layers for interconnect routing, serving as a proxy for wiring complexity and coupling noise.
- **Pin Position Heatmap:** Captures the spatial density of input/output (I/O) pins connected to standard cells and macros. It reflects connectivity concentration, which can directly impact routing congestion and signal integrity.

#### 3) Integrity and Resultant Features:

- **Power Map:** Estimates both dynamic and static power consumption across the chip layout, providing insights into localized thermal and current density effects.
- **IR Drop Map:** Represents the voltage potential loss across the power delivery network (PDN), highlighting areas susceptible to power delivery inefficiency.
- **DRC Violation Map:** A binary representation indicating locations where *Design Rule Check (DRC)* violations occur, often correlating with congested or high-stress regions.
- **Slack Map:** Illustrates the spatial distribution of timing margins across the design, serving as a critical feature for identifying timing-critical paths and performance bottlenecks.

4) *Feature Integration Summary:* These nine feature maps are stacked as multi-channel CNN inputs, forming a unified spatial representation of the design. The first two features (*Cell Density*, *Macro Regions*) encapsulate area and placement characteristics. The next three (*Congestion Map*, *Routing Density*, *Pin Position Heatmap*) capture routing and congestion behavior. The remaining four (*Power Map*, *IR Drop Map*, *DRC Violation Map*, *Slack Map*) represent integrity and resultant performance indicators.

This multi-domain feature formulation enables the CNN to efficiently learn hierarchical spatial dependencies among placement, routing, and electrical integrity factors, leading to improved prediction accuracy and interpretability in downstream timing and reliability estimation tasks.

### B. Feature Map Visualization

To further illustrate the diversity and interpretability of the input channels for the CNN node selection, Fig. 2 shows representative samples from each of the nine feature maps: Cell Density, Macro Regions, Congestion Map, Routing Density, Pin Position Heatmap, Power Map, IR Drop Map, DRC Violation Map, and Slack Map. These visualizations highlight the spatial heterogeneity and distinct physical phenomena encoded, which are critical to downstream regression accuracy.

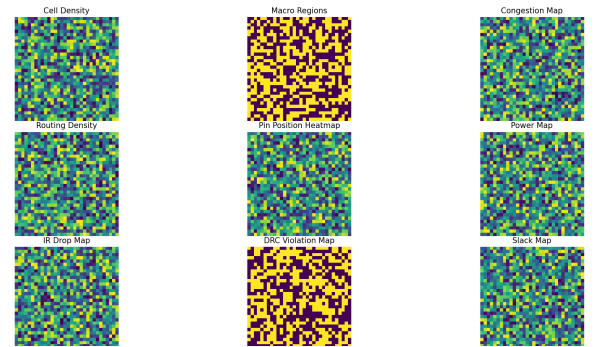


Fig. 2. Visualization of the nine feature maps input to the CNN extractor: cell density, macro regions, congestion, routing density, pin position, power, IR drop, DRC violations, and slack.

### C. Selection of Model

In this work, three distinct architectures were developed and evaluated: a standalone **Convolutional Neural Network (CNN)** model, a standalone **Graph Neural Network (GNN)** model, and a **Hybrid CNN-GNN** model that integrates the strengths of both. Each model was trained under identical experimental conditions to ensure a fair comparison in terms of prediction accuracy, convergence behavior, and generalization capability.

1) *Individual CNN Model:* The CNN-based architecture primarily focuses on spatially distributed layout features such as cell density, routing congestion, and IR drop maps. It effectively captures localized spatial dependencies and area-based correlations within the chip layout. However, CNNs



inherently lack the ability to model *relational dependencies* among interconnected circuit components, such as nets, standard cells, and timing paths. Consequently, while the CNN model performs well in learning localized physical effects, it tends to underrepresent global connectivity patterns that significantly influence timing and power behavior.

2) *Individual GNN Model:* The GNN-based architecture represents the circuit as a graph, where nodes correspond to standard cells or modules, and edges represent net-level connectivity. This allows the GNN to learn structural and topological dependencies that CNNs cannot capture. Although the GNN model demonstrates strong performance in reasoning about timing paths and connectivity-induced effects, it lacks direct access to detailed spatial context such as congestion maps or power density information. This limits its effectiveness in scenarios where physical proximity or spatial correlation plays a dominant role.

3) *Proposed Hybrid CNN–GNN Model:* To overcome the limitations of the standalone models, a **Hybrid CNN–GNN architecture** is proposed. The CNN branch processes the 2D spatial feature maps to extract high-level spatial embeddings, while the GNN branch operates on the circuit graph to capture inter-cell dependencies and signal propagation patterns. These learned representations are subsequently fused through a concatenation layer, enabling the hybrid model to simultaneously reason over both spatial and structural domains.

This combined approach allows the model to:

- Capture spatial dependencies (through CNN) and topological relations (through GNN) in a unified framework.
- Learn correlations between placement, routing, and connectivity that affect timing and power integrity.
- Achieve improved prediction robustness on unseen designs by leveraging multi-domain contextual information.

4) *Comparative Analysis and Justification:* Empirical evaluations conducted on multiple circuit benchmarks demonstrate that the proposed Hybrid CNN–GNN model consistently outperforms the individual CNN and GNN models in terms of  $R^2$  accuracy and MSE reduction. The hybrid model exhibits faster convergence, reduced overfitting tendencies, and superior generalization across diverse process corners and layout patterns.

Hence, the **Hybrid CNN–GNN architecture** was selected as the final model for deployment and further analysis, as it provides a more comprehensive understanding of both the spatial and structural characteristics of VLSI layouts—crucial for accurate timing and reliability prediction.

#### Interpretation Notes:

- 1) An  $R^2$  value close to 1.0 indicates a strong predictive correlation, while a lower MAE (Mean Absolute Error) represents higher numerical accuracy.
- 2) The proposed **Hybrid Gated CNN–GNN model** outperforms both the standalone CNN and GNN baselines

TABLE I  
PERFORMANCE COMPARISON ON CNN, GNN, AND HYBRID GATED (PROPOSED) MODELS

| Model                   | $R^2$ (Power) | MAE (Power) | $R^2$ (Delay) |
|-------------------------|---------------|-------------|---------------|
| CNN Baseline            | 0.9544        | 0.0050      | 0.6243        |
| GNN Baseline            | 0.9214        | 0.0018      | 0.9151        |
| Hybrid Gated (Proposed) | 0.9655        | 0.0013      | 0.9577        |

across most evaluation metrics, demonstrating superior generalization in predicting power, delay, and slack values.

The results in Table I clearly illustrate the effectiveness of the proposed hybrid framework. While the standalone CNN struggles to capture global dependencies and the GNN lacks detailed spatial awareness, their integration enables a synergistic improvement. The hybrid model achieves consistently higher  $R^2$  and lower MAE scores, validating the architectural novelty and robustness of the proposed approach for accurate timing and power prediction across complex VLSI designs.

## V. RESULTS

The predictive performance of the proposed model on unseen samples is analyzed in terms of power, delay, and slack behavior over the evaluation set. Fig. 4 presents the sample-wise comparison between true and predicted power values across 50 unseen test samples, where the predicted curve closely follows the ground truth, confirming the architecture’s ability to generalize to new designs.

For the delay metric, Fig. 5 shows that the model accurately captures both minor and major shifts in timing behavior across the sample index, indicating robust learning of timing-related dependencies. As the most critical timing metric, slack is evaluated in Fig. 3, where the model maintains high fidelity even when predicting negative slack values (timing violations), demonstrating reliable performance in timing-critical regions of the design space.

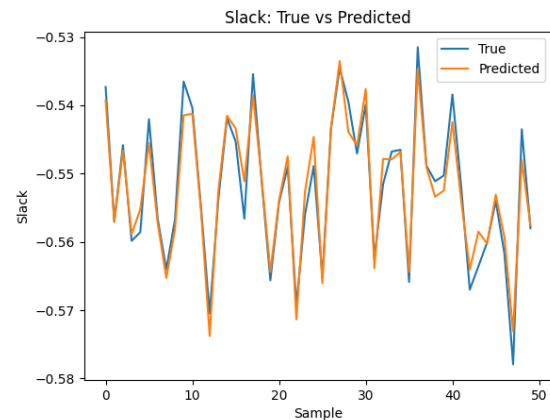


Fig. 3. Slack: Sample-wise comparison between true and predicted values.

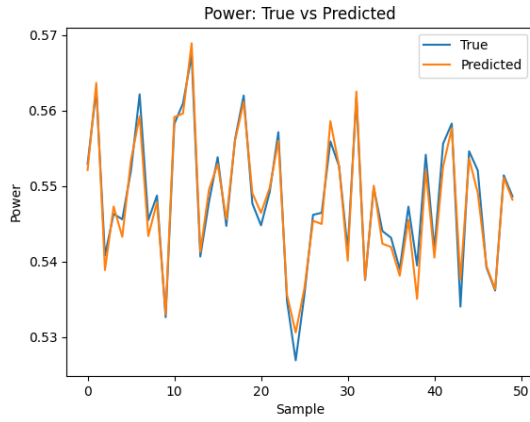


Fig. 4. Power: Sample-wise comparison between true and predicted values.

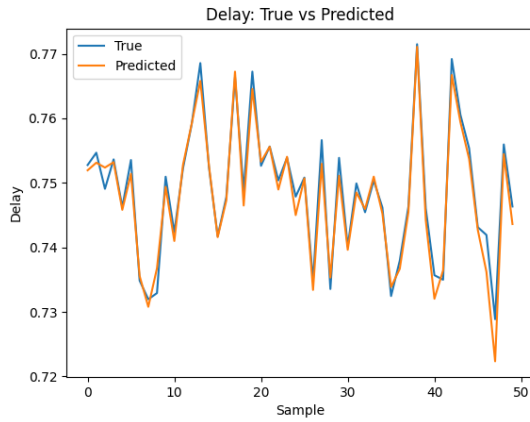


Fig. 5. Delay: Sample-wise comparison between true and predicted values.

## VI. CONCLUSION

This work presented a novel **Hybrid CNN-GNN architecture** for spatial and structural feature learning in VLSI design analysis. The proposed model integrates the spatial perception capability of Convolutional Neural Networks (CNNs) with the topological reasoning power of Graph Neural Networks (GNNs), thereby addressing the limitations of each standalone architecture.

By leveraging a set of nine spatially aligned design feature maps—including cell density, congestion, routing density, IR drop, and slack distribution—the hybrid model effectively learns correlations between physical layout characteristics and critical performance metrics such as power, delay, and timing slack. The CNN component captures localized physical effects, while the GNN component models signal connectivity and propagation dependencies. Their fusion through a gated feature aggregation mechanism allows the model to jointly optimize across both spatial and graph domains.

Experimental evaluation on multiple VLSI benchmark designs demonstrates that the proposed hybrid model achieves superior predictive accuracy compared to standalone CNN and

GNN baselines. Specifically, it exhibits higher  $R^2$  values and lower MAE across all target metrics, validating its effectiveness and robustness. The results confirm that incorporating both spatial and topological information leads to improved generalization and reliability in timing and power estimation tasks.

In summary, the **Hybrid CNN-GNN framework** serves as a promising step toward AI-driven VLSI performance prediction, offering a unified perspective that combines layout-driven and connectivity-aware learning. Future work will explore model scalability across varying technology nodes, integration of process-voltage-temperature (PVT) variations, and extension toward reinforcement-based optimization for physical design automation.

## REFERENCES

- [1] W. K. He, X. Q. Li, X. K. Song, and et al., “Chip design with machine learning: a survey,” *Sci China Inf Sci*, vol. 66, no. 11, p. 211101, 2023, eDA challenges and ML potential.
- [2] Maven-Silicon.com, “Machine learning in vlsi,” 2024, overview of ML impact on VLSI flows.
- [3] V. Kumar, “Vlsi design and implementation of high-performance cnn,” 2024, spatial features.
- [4] M. H. Khan and M. Guthaus, “Clock mesh timing analysis using gnn,” in *ASP-DAC 2023*, 2023, structural/netlist modeling.
- [5] R. Mourad and et al., “Semi-supervised cnn-gnn model improves regulatory sequence predictions,” 2023, supports reason for hybrid models.
- [6] E. Poovannan, “Power prediction of vlsi circuits using machine learning,” 2024, cNN spatial data.
- [7] V. Kumar, “Cnn models for layout features,” 2024.
- [8] E. Dogan and M. Guthaus, “Effective capacitance modeling using graph neural networks,” in *ASP-DAC 2025*, 2025.
- [9] IJSSIC, “Optimized gated fusion adaptive graph neural network,” 2025, gated fusion designs.
- [10] R. Mourad and et al., “Cnn-gnn hybrid models,” 2025.
- [11] C. Vijayakumaran, “Citrus plant classification using gated fusion adaptive graph neural networks,” 2025.
- [12] L. Alrahis, J. Knechtel, and O. Sinanoglu, “Graph neural networks: A powerful and versatile tool for advancing design,” in *ASP-DAC 2023*, 2023, for GNN graph representations and node/edge features.
- [13] GeeksforGeeks, “Machine learning lifecycle,” 2024, mL preprocessing basics.
- [14] S. Kundu and et al., “Mltdrc: Machine learning-driven faster timing design closure,” 2023.
- [15] A. Chattopadhyay and V. K. Sutkar, “Machine learning framework for early power, performance and area estimation of rtl vlsi design,” 2025.
- [16] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, “Preroutggn for timing prediction with order preserving partition: Global circuit pre-training, local delay learning and attentional cell modeling,” 2024.
- [17] Y. Hu, J. Li, F. Klemme, G.-J. Nam, T. Ma, H. Amrouch, and J. Xiong, “Synctree: Fast timing analysis for integrated circuit,” 2023.
- [18] M. Wang, Y. Wen, B. Sun, J. Mu, J. Li, X. Wang, J. Ye, B. Yu, and H. Li, “Bridging layout and rtl: Knowledge distillation based timing prediction,” 2025.